Comparative Analysis of Finite State Machines and Behavior Trees for Stealth Game AI

Shadow Protocol: A Stealth Game Case Study

Huy Gia Nguyen
22044312

UFCFXK-30-3 Digital Systems Project
University of the West of England, Bristol
School of Computing and Creative Technologies

April 2026

Code Repository: https://github.com/HuyNguyen2307/shadow-protocol

Abstract

This project presents a comparative analysis of Finite State Machines (FSM) and Behavior Trees (BT) for implementing artificial intelligence in stealth video games. The stealth genre presents unique AI requirements including graduated detection systems, multi-sensory awareness, coordinated threat responses, and believable investigation behaviors that challenge both architectural approaches differently.

Using Unity 2022 and C#, a functional stealth game prototype titled Shadow Protocol was developed, implementing both AI architectures with identical sensory subsystems to enable methodologically sound comparison. The shared architecture includes a VisionSensor with graduated DetectionMeter, HearingSensor with spatial audio processing, AIMemory for persistent state tracking, and AlertSystem enabling multi-agent communication across eight concurrent guards.

Quantitative analysis revealed that the FSM implementation required 1,010 lines of code compared to 1,326 lines for the Behavior Tree, representing a 31% increase. However, the BT's reusable core framework (529 lines) means game-specific code is actually 21% smaller. Runtime metrics collected during an 87.2-second test session recorded 70 state transitions, with PATROL (40%) and INVESTIGATE (41%) being most frequent, confirming both hearing and patrol systems function correctly. The Behavior Tree demonstrated superior extensibility, requiring approximately 15 minutes to add new behaviors compared to 45 minutes for equivalent FSM modifications.

The findings indicate that FSM remains preferable for projects with well-defined, stable requirements, while Behavior Trees offer significant advantages for complex systems requiring frequent iteration or code reuse across projects. Both architectures achieved functionally equivalent gameplay outcomes, confirming that architectural choice should be guided by development process considerations rather than runtime performance concerns.

Keywords: Finite State Machine, Behavior Tree, Game AI, Stealth Games, Unity, Decision Architecture

Acknowledgements

I would like to express my sincere gratitude to my project supervisor for their invaluable guidance, constructive feedback, and continuous support throughout this project. Their expertise in game development and artificial intelligence significantly influenced the direction and quality of this research. I also thank the UWE Bristol School of Computing and Creative Technologies for providing necessary resources and academic environment conducive to this investigation. I acknowledge the use of generative AI tools in an assistive capacity for drafting and structuring certain sections of this report, as permitted under module guidelines. All AI-generated content was critically reviewed and substantially modified to ensure accuracy and academic integrity.

Table of Contents

Abstract 2
Acknowledgements 3
Table of Contents 4
1. Introduction 6
  1.1 Background and Motivation 6
  1.2 Research Question 6
  1.3 Aims and Objectives 7
  1.4 Scope and Deliverables 7
2. Literature Review 9
  2.1 Finite State Machines in Game AI 9
  2.2 Behavior Trees: Origins and Principles 9
  2.3 Gap in Literature 10
3. Design and Methodology 11
  3.1 Development Environment and Tools 11
  3.2 Shared Sensory Architecture 11
  3.3 FSM Implementation 12
  3.4 Behavior Tree Implementation 12
  3.5 Metrics Collection Methodology 12
4. Results and Analysis 14
  4.1 Code Complexity Metrics 14
  4.2 Runtime Performance Metrics 14
  4.3 Development Experience Analysis 15
  4.4 Discussion and Synthesis 15
5. Project Evaluation 17
  5.1 Achievement of Objectives 17
  5.2 Strengths 17
  5.3 Limitations 17
  5.4 Critical Reflection 18
6. Further Work and Conclusion 19
  6.1 Recommendations for Further Work 19
  6.2 Conclusion 19
References 21
Appendix A: System Architecture 22
  A.1 FSM State Transition Diagram 22
  A.2 Behavior Tree Structure 22

A.3 Project File Structure 22

1. Introduction

1.1 Background and Motivation

Artificial intelligence in video games has undergone remarkable evolution since the deterministic rule-based systems of early arcade titles. Contemporary games demand sophisticated AI that creates believable, challenging, and engaging player experiences while maintaining computational efficiency suitable for real-time execution at consistent frame rates. This requirement is particularly pronounced in the stealth game genre, where enemy AI must demonstrate convincing environmental awareness, nuanced investigation behaviors, graduated threat responses, and coordinated multi-agent reactions that players perceive as both challenging and fair (Millington & Funge, 2019).

The commercial success of stealth franchises including Metal Gear Solid, Hitman, Splinter Cell, and Dishonored has established clear player expectations for enemy AI behavior. Guards should patrol believably along defined routes, react appropriately to both visual and auditory stimuli with contextual responses, investigate disturbances methodically without appearing scripted, coordinate effectively with allies during alert states, and return to normal behavior patterns when threats are not confirmed. Failure to meet these expectations results in AI that players perceive as either frustratingly omniscient or unrealistically oblivious, both of which undermine the core stealth gameplay experience.

Two dominant architectural patterns have emerged for implementing such sophisticated AI systems: Finite State Machines (FSM) and Behavior Trees (BT). Both approaches have demonstrated success in commercial game development, with notable implementations including the Halo series' Behavior Trees and numerous indie titles employing well-designed FSMs. However, these architectures present distinct trade-offs in terms of implementation complexity, runtime flexibility, maintainability over extended development cycles, and suitability for different team structures and project requirements.

1.2 Research Question

This project addresses the following primary research question:

"What are the trade-offs between Finite State Machines and Behavior Trees when implementing stealth game AI, considering development complexity, code maintainability, runtime flexibility, and resulting player experience?"

Secondary questions guiding the investigation include: How do quantitative code complexity metrics compare between equivalent implementations? What are the relative development time investments required for adding new behaviors to established systems? Do observable differences in runtime performance exist under typical gameplay conditions with multiple concurrent AI agents?

1.3 Aims and Objectives

The primary aims of this project are:

- Design and implement a functional stealth game prototype demonstrating core genre mechanics including patrol behavior, graduated detection, pursuit, search, and alert-response systems
- Implement enemy AI using both FSM and Behavior Tree architectures, with both systems sharing identical sensory subsystems to enable methodologically valid comparison
- Develop comprehensive sensory systems including vision with graduated detection meters, hearing with spatial audio processing and distance attenuation, and multi-agent communication through a coordinated alert system
- Collect and analyze both quantitative metrics (lines of code, state transitions, time distributions) and qualitative observations (development experience, debugging complexity)
- Provide evidence-based recommendations for game developers selecting between these architectural approaches for different project contexts

1.4 Scope and Deliverables

The project scope encompasses development of a third-person stealth game prototype titled Shadow Protocol using Unity 2022 LTS and C#. Key deliverables include: the functional game prototype with both AI implementations selectable at runtime; shared sensory components (VisionSensor, HearingSensor, AIMemory, AlertSystem); a metrics collection system for runtime data gathering; this comparative analysis report documenting methodology and findings; and a complete

code repository containing all source files. The comparison focuses specifically on decision-making architecture rather than perception system differences, as both implementations intentionally share identical sensory components to isolate the architectural comparison.

2. Literature Review

2.1 Finite State Machines in Game AI

Finite State Machines represent one of the earliest and most enduring approaches to game AI implementation. An FSM consists of a finite set of discrete states, conditional transitions between those states triggered by specific events or conditions, and actions associated with each state that determine agent behavior (Bourg & Seemann, 2004). The fundamental elegance of FSMs lies in their conceptual simplicity: an agent occupies exactly one state at any given time, and the current state fully determines both the available behaviors and the conditions that might trigger state changes.

Classic games including Pac-Man demonstrated remarkably effective FSM implementations where ghost characters transitioned between chase, scatter, and frightened states based on well-defined game events (Pittman, 2011). The transparency of FSM logic facilitates debugging, as developers can easily trace the sequence of states and transitions leading to any observed behavior. This predictability makes FSMs particularly suitable for smaller teams or projects where rapid iteration on core mechanics takes priority over architectural sophistication.

However, FSMs present significant scalability challenges as behavioral complexity increases. The number of required states and transitions grows exponentially as new behaviors and interaction conditions are added, a phenomenon termed state explosion by researchers (Isla, 2005). Adding a single new behavior may require defining transitions to and from every existing state, creating substantial maintenance burden that compounds over extended development cycles.

2.2 Behavior Trees: Origins and Principles

Behavior Trees emerged from robotics research and gained widespread recognition in game development following successful implementation in Bungie's Halo 2 (Isla, 2005). A Behavior Tree is a hierarchical, tree-structured model of nodes that control agent decision-making through systematic tree traversal from root to leaves. The architecture defines several node categories: composite nodes (Selectors and Sequences) that determine child evaluation order, decorator nodes that modify child behavior, and leaf nodes comprising conditions for testing world state and actions for executing behaviors (Colledanchise & Ögren, 2018).

Selector nodes implement priority-based decision making by evaluating children sequentially and succeeding when any child succeeds, effectively implementing logical OR operations. Sequence nodes require all children to succeed sequentially, implementing logical AND operations useful for multi-step behaviors. The key architectural advantage lies in modularity: behaviors encapsulated in subtrees can be designed, tested, and reused across different agents or even different projects. Adding new behaviors involves inserting branches at appropriate priority levels without modifying existing node implementations (Simpson, 2014).

2.3 Gap in Literature

Academic literature directly comparing FSM and BT implementations in equivalent contexts remains surprisingly limited. Florez-Puga et al. (2009) noted the differing maintenance profiles but did not provide implementation-based comparison. Simpson (2014) recommended Behavior Trees for iterative development projects but based conclusions primarily on theoretical analysis rather than empirical measurement. This project contributes implementation-based comparison using identical sensory systems within a complete, functional prototype, generating both quantitative metrics and qualitative observations to inform practical development decisions.

3. Design and Methodology

3.1 Development Environment and Tools

The project was developed using Unity 2022 LTS (Long Term Support) with C# as the primary programming language. Unity was selected for its robust game development ecosystem, comprehensive documentation, built-in NavMesh pathfinding system providing reliable agent navigation, and widespread industry adoption ensuring relevance of findings. The NavMesh system provided consistent navigation behavior across both AI implementations, effectively eliminating pathfinding differences as a confounding variable in the comparison. Development followed an iterative methodology with weekly supervisor meetings providing feedback checkpoints and regular testing cycles identifying issues early in the development process.

3.2 Shared Sensory Architecture

To ensure methodologically valid comparison, all sensory systems were implemented as shared components completely independent of decision-making logic. This architectural decision, made early in the development process before either AI implementation began, proved retrospectively crucial for enabling meaningful comparison. Both AI architectures receive identical perceptual inputs, ensuring any behavioral differences arise solely from decision-making architecture rather than perception differences.

The VisionSensor component implements field-of-view detection with configurable range (8 meters default) and angle (90 degrees). Critically, it features a graduated DetectionMeter that accumulates continuously when the player remains within line-of-sight, creating gradual awareness escalation rather than instant binary detection. This mechanic, common in modern stealth games, gives players feedback through UI indicators and brief windows to break line-of-sight before full detection occurs. The sensor performs raycasting verification to prevent detection through walls and obstacles.

The HearingSensor processes sound events generated by the PlayerNoiseSystem component, which emits audio events based on player movement speed, surface type, and actions. Sound propagation includes distance-based attenuation and wall occlusion calculations. The sensor maintains a suspicion meter that accumulates

with repeated sounds, triggering investigation behaviors when thresholds are exceeded.

The AIMemory component provides persistent storage for last-known player positions with timestamps, enabling search behaviors when the player breaks visual contact. The AlertSystem manages inter-agent communication through a singleton coordinator pattern, enabling guards who detect the player to broadcast alerts to nearby allies within configurable range (20 meters default). Alert propagation supports multiple modes: range-based for localized alerts, line-of-sight for realistic communication, and global for high-alert scenarios.

3.3 FSM Implementation

The FSM implementation (EnemyAI_Advanced.cs, 1,010 lines) defines six discrete states representing the complete behavioral repertoire: PATROL for standard waypoint-following behavior, INVESTIGATE for responding to audio stimuli, SUSPICIOUS for partial visual detection, CHASE for active pursuit when detection reaches 100%, SEARCH for methodically examining the player's last-known position, and RESPOND_ALERT for coordinated response to alerts from other guards. The implementation follows a state pattern with explicit Enter, Update, and Exit methods for each state, centralizing transition logic while maintaining clear separation between state behaviors.

3.4 Behavior Tree Implementation

The Behavior Tree comprises two components: a reusable core framework (BTCore.cs, 529 lines) and the game-specific tree implementation (EnemyAI_BT.cs, 797 lines). The core framework defines abstract classes for Selector, Sequence, Condition, and Action nodes with proper SUCCESS/FAILURE/RUNNING status propagation according to standard BT semantics. The game-specific tree implements priority-based behavior selection: CHASE (highest priority), RESPOND_ALERT, SEARCH, INVESTIGATE, and PATROL (lowest priority, serving as fallback behavior). This priority ordering ensures urgent behaviors preempt routine activities appropriately.

3.5 Metrics Collection Methodology

A custom AIMetricsCollector component gathers runtime data including state transition counts per state, cumulative time spent in each state across all agents, and player detection events. Code complexity metrics were gathered through manual

line counting and structural analysis. Test sessions were conducted with eight concurrent AI agents patrolling a multi-room warehouse environment while the player navigated attempting to avoid detection, simulating realistic gameplay conditions.

4. Results and Analysis

4.1 Code Complexity Metrics

Table 1 presents quantitative code complexity metrics comparing the two implementations. These measurements provide objective comparison of implementation overhead and structural characteristics.

Metric	FSM	Behavior Tree
Total Lines of Code	1,010	1,326 (797+529)
Core Framework Lines	N/A (integrated)	529 lines
Game-Specific Code	1,010 lines	797 lines
States/Root Branches	6 states	5 branches
Total Decision Nodes	6	19 nodes
Cyclomatic Complexity	Medium	Low (modular)

The Behavior Tree requires 31% more total code than the FSM implementation; however, this difference is primarily attributable to the reusable core framework component. When considering only game-specific code, the BT implementation is actually 21% smaller than the monolithic FSM, reflecting the cleaner separation of infrastructure concerns from application-specific logic. The modular BT structure also facilitates unit testing of individual nodes, though this capability was not extensively utilized in this project.

4.2 Runtime Performance Metrics

Runtime data was collected during an 87.2-second test session with eight concurrent AI agents patrolling the test environment. Table 2 presents detailed state transition analysis from this recorded gameplay session.

State	Transitions	Percentage	Time Spent
PATROL	28	40.0%	522.3s
INVESTIGATE	29	41.4%	103.0s
SUSPICIOUS	8	11.4%	16.5s
CHASE	3	4.3%	9.0s
SEARCH	2	2.9%	16.0s
RESPOND_ALERT	0	0.0%	0.0s
Total	70	100%	87.2s session

Note: Time spent values are aggregated across all eight simultaneously active agents, explaining why totals exceed the session duration. The high INVESTIGATE transition proportion (41.4%) indicates the hearing system is functioning effectively, with guards responding appropriately to player movement sounds. The relatively low

CHASE rate (4.3%) confirms the stealth mechanics provide adequate concealment opportunities for skilled players. The absence of RESPOND_ALERT transitions indicates no guard achieved full detection to trigger alerts in this particular test session. Both implementations maintained frame rates above 60 FPS throughout testing, indicating negligible performance impact from either architecture.

4.3 Development Experience Analysis

Table 3 summarizes qualitative assessment based on development experience throughout the project. Ratings reflect subjective evaluation but are grounded in specific development observations.

Criterion	FSM Rating	BT Rating
Ease of Understanding	★★★★★ Easy	★★★☆☆ Medium
Initial Implementation Speed	★★★★☆ Good	★★★☆☆ Medium
Extensibility	★★☆☆☆ Hard	★★★★★ Easy
Long-term Maintainability	★★★☆☆ Medium	★★★★☆ Good
Debugging Transparency	★★★★☆ Easy	★★★☆☆ Medium
Code Reusability	★★☆☆☆ Low	★★★★★ High

Initial FSM implementation required approximately 4 hours of development time versus 6 hours for the Behavior Tree including core framework development. However, the process of adding the RESPOND_ALERT behavior later in development revealed the significant extensibility difference: the FSM modification required approximately 45 minutes of changes across multiple methods with careful consideration of transition conditions, while the BT modification required only 15 minutes to add a new branch at the appropriate priority level without touching existing code.

4.4 Discussion and Synthesis

Results support context-dependent architectural recommendations rather than universal preference for either approach. FSM demonstrated clear advantages in initial implementation accessibility, debugging transparency through explicit state visualization, and marginally lower memory overhead, making it suitable for projects with well-understood, stable requirements and smaller development teams. Behavior Trees demonstrated compelling advantages in extensibility, long-term maintainability, and code reusability across projects, aligning well with iterative development methodologies where requirements evolve throughout the development cycle.

For Shadow Protocol's specific requirements, both architectures achieved functionally equivalent player experience outcomes. Guards patrol believably, investigate sounds appropriately, detect and pursue the player with graduated escalation, and search methodically when contact is lost. The observable AI behavior is indistinguishable between implementations from a player perspective, confirming that the choice between architectures should be guided by development process considerations rather than gameplay outcome requirements.

5. Project Evaluation

5.1 Achievement of Objectives

The project successfully achieved all stated objectives within the allocated timeframe. A fully functional stealth game prototype was developed demonstrating complete patrol, detection, investigation, pursuit, search, and alert-response mechanics functioning correctly. Both AI architectures were fully implemented sharing identical sensory systems, enabling methodologically valid comparison. The metrics collection system provided quantitative data supporting the analysis, and this report documents findings for practical application by other developers facing similar architectural decisions.

5.2 Strengths

- Comprehensive Implementation: Both architectures were fully implemented with identical behavioral capabilities including the multi-agent communication system
- Fair Comparison Methodology: The shared sensor architecture enabled valid architectural comparison by eliminating perception differences as confounding variables
- Advanced Feature Set: Graduated detection, spatial hearing with occlusion, persistent memory, and inter-guard communication exceeded typical academic prototypes
- Practical Metrics: Runtime data was collected from actual gameplay sessions with realistic agent counts rather than synthetic benchmarks
- Reproducible Results: All source code is provided in the accompanying repository enabling verification and extension of findings

5.3 Limitations

- Sample Size: The single extended session (87.2 seconds) provides indicative rather than statistically robust data; additional sessions would strengthen conclusions
- Single Developer Perspective: Development time comparisons reflect one developer's learning curve and prior experience, limiting generalizability
- Platform Specificity: Testing was conducted exclusively in Unity; results may differ in other game engines with different architectural patterns

- RESPOND_ALERT Coverage: Zero alert transitions in the recorded session limits quantitative analysis of this specific behavior's performance
- Player Behavior Variability: Test sessions involved a single play style; different player approaches might yield different state distributions

5.4 Critical Reflection

The decision to implement shared sensors before beginning either AI architecture proved retrospectively crucial for the project's validity. Had sensors been developed alongside the first AI implementation, later extraction for sharing would have required significant refactoring and introduced potential inconsistencies. This early architectural decision, made based on supervisor guidance during project planning, enabled the fair comparison that forms this project's core methodological contribution.

The iterative development approach revealed several assumptions requiring revision during implementation. Initial hearing system parameters caused suspicion to decay too rapidly, preventing investigation triggers from activating reliably. Regular testing cycles identified and resolved such tuning issues progressively, highlighting the importance of continuous playtesting even for AI-focused development. Weekly supervisor meetings provided valuable scoping decisions that prevented feature creep while ensuring sufficient depth for meaningful comparison.

6. Further Work and Conclusion

6.1 Recommendations for Further Work

- Hybrid Architecture Investigation: Combining FSM for high-level state management with BT for detailed behavior selection within states may capture advantages of both approaches
- Extended Statistical Testing: Longer test sessions with multiple testers across diverse scenarios would provide statistically robust validation of findings
- Formal User Studies: Controlled player testing with blindfolded architecture selection would evaluate whether any perceivable differences exist from the player perspective
- Alternative Architecture Comparison: Extending comparison to include Goal-Oriented Action Planning (GOAP) and Utility AI would provide broader architectural guidance
- Performance Profiling: Detailed CPU and memory profiling under stress conditions with larger agent counts would quantify performance differences more precisely

6.2 Conclusion

This project investigated the practical trade-offs between Finite State Machines and Behavior Trees for stealth game AI through developing Shadow Protocol with both architectural approaches sharing identical sensory systems. Runtime metrics from an 87.2-second test session with eight concurrent agents recorded 70 state transitions across six behavioral states, with PATROL (40%) and INVESTIGATE (41%) being most frequent, demonstrating both architectures functioning correctly under realistic gameplay conditions.

Key findings indicate that FSM remains a valid choice for projects with stable, well-understood requirements, offering implementation simplicity and debugging transparency that benefit smaller teams or shorter development cycles. Behavior Trees demonstrate clear superiority for projects requiring iterative development, frequent behavioral modifications, or code reuse across multiple projects. The 31% additional BT code is offset by 21% smaller game-specific implementation and significantly improved extensibility, with new behavior implementation requiring approximately one-third the time compared to FSM modification.

For stealth games specifically, both architectures achieve equivalent player experience outcomes when properly implemented. Guards behave indistinguishably from a player perspective regardless of underlying decision architecture. The choice between approaches should therefore be guided by development process considerations including team expertise with each pattern, expected frequency of behavioral iteration during development, and long-term maintenance and reuse requirements. This project contributes practical, implementation-based evidence addressing the gap between theoretical architectural discussions and actionable development guidance.

References

Bourg, D.M. and Seemann, G. (2004) AI for Game Developers. Sebastopol, CA: O'Reilly Media.

Champandard, A.J. (2007) 'Understanding Behavior Trees', AiGameDev.com. Available at: http://aigamedev.com/open/article/behavior-trees-part1/

Colledanchise, M. and Ögren, P. (2018) Behavior Trees in Robotics and AI: An Introduction. Boca Raton, FL: CRC Press.

Florez-Puga, G. et al. (2009) 'Query-enabled behavior trees', IEEE Transactions on Computational Intelligence and AI in Games, 1(4), pp. 298-308.

Isla, D. (2005) 'Handling Complexity in the Halo 2 AI', Game Developers Conference 2005, San Francisco, CA.

Millington, I. and Funge, J. (2019) Artificial Intelligence for Games. 3rd edn. Boca Raton, FL: CRC Press.

Pittman, J. (2011) 'The Pac-Man Dossier', Gamasutra. Available at: https://www.gamedeveloper.com/design/the-pac-man-dossier

Simpson, C. (2014) 'Behavior trees for AI: How they work', Gamasutra. Available at: https://www.gamedeveloper.com/programming/behavior-trees-for-ai-how-they-work

Appendix A: System Architecture

A.1 FSM State Transition Diagram

```
PATROL ──(heard sound)──────────────────> INVESTIGATE
PATROL ──(DetectionMeter > 0.3)──────────> SUSPICIOUS
PATROL ──(received alert from ally)───> RESPOND_ALERT
INVESTIGATE ──(nothing found)──────────> PATROL
INVESTIGATE ──(DetectionMeter ≥ 1.0)──> CHASE
SUSPICIOUS ──(DetectionMeter ≥ 1.0)───> CHASE
SUSPICIOUS ──(detection decay)──────────> PATROL
CHASE ──(lost sight for 2+ seconds)───> SEARCH
SEARCH ──(timeout after 5 seconds)─────> PATROL
SEARCH ──(player detected)─────────────> CHASE
RESPOND_ALERT ──(scan complete)────────> PATROL
```

A.2 Behavior Tree Structure

```
Root [Selector] - Priority-based behavior selection
├── CHASE [Sequence]: DetectionMeter ≥ 1.0
│   ├── Condition: IsPlayerFullyDetected
│   ├── Action: SetChaseSpeed
│   └── Action: PursuePlayer (RUNNING while chasing)
├── RESPOND_ALERT [Sequence]: Alert received from ally
│   ├── Condition: HasPendingAlert
│   ├── Action: MoveToAlertPosition
│   └── Action: ScanAlertArea
├── SEARCH [Sequence]: Lost visual contact
│   ├── Condition: ShouldSearch
│   ├── Action: MoveToLastKnownPosition
│   └── Action: ScanArea (360° rotation)
├── INVESTIGATE [Sequence]: Heard suspicious sound
│   ├── Condition: HasInvestigateTarget
│   ├── Action: MoveToSoundSource
│   └── Action: LookAroundAtSound
└── PATROL [Sequence]: Default fallback behavior
    ├── Action: MoveToNextWaypoint
    └── Action: WaitAtWaypoint
```

A.3 Project File Structure

- EnemyAI_Advanced.cs (FSM): 1,010 lines implementing 6 states with explicit transitions
- BTCore.cs: 529 lines providing reusable Selector, Sequence, Condition, and Action node classes
- EnemyAI_BT.cs: 797 lines implementing game-specific behavior tree with 5 priority branches
- VisionSensor.cs: Graduated detection with configurable range, angle, and accumulation rate

- HearingSensor.cs: Spatial audio processing with distance attenuation and wall occlusion
- AlertSystem.cs: Singleton coordinator enabling multi-agent communication with configurable propagation modes
- AIMemory.cs: Persistent storage for last-known player positions with timestamps
- AIMetricsCollector.cs: Runtime data collection for state transitions and time distributions

Complete source code is available in the linked repository with documentation and setup instructions.